

Pill AI Pro

A Project Report

Submitted by:

Vinit Shah (Symbiosis International University, 25030142025)

in partial fulfillment for the award of the degree

of

M.SC. COMPUTER APPLICATIONS

In

DATA SCIENCE

at

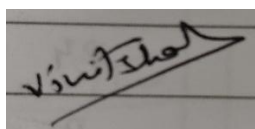
**SYMBIOSIS INSTITUTE OF COMPUTER STUDIES AND RESEARCH, PUNE,
INDIA**

**AFFILIATED TO SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)
(INDIA)**

March 2026

DECLARATION

I hereby declare that the project entitled “_____Pill AI Pro_____” submitted for the M.Sc. Computer Application degree is my original work and the project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

A handwritten signature in black ink, appearing to read 'Vinit Shah', is written over a horizontal line.

Signature of the Student

Place: SICSR

Date: 23/03/2026

CERTIFICATE

This is to certify that the project titled “_____ Pill AI Pro _____” is the bona fide work carried out by __Vinit Shah__, a student of M.Sc. Computer Application of Symbiosis Institute of Computer Studies and Research, Pune India, affiliated to Symbiosis International (Deemed University) during the academic year 2025-26, in partial fulfillment of the requirements for the award of the degree of M.Sc. Computer Application.



Signature of the Guide

Place: SICSR

Date: 23/03/2026



SYMBIOSIS INSTITUTE OF COMPUTER STUDIES & RESEARCH

Symbiosis International (Deemed University)

Established under section 3 of the UGC Act 1956

Re-accredited by NAAC with 'A++' grade | Awarded Category-I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Date: 24/03/2026

COMPLETION CERTIFICATE

This is to certify that the Dissertation report titled **Pill AI Pro** is the bonafide work **Vinit Shah** PRN: 25030142025, MSc (CA) Batch 2025-2027 at Symbiosis Institute of Computer Studies and Research (SICSR) who carried out the “Dissertation” work under my supervision. He has completed the Dissertation during dated from **16/01/2026 to 24/03/2026.**

Dr. Prakash Raut

Project Guide

ACKNOWLEDGEMENT

I am wholeheartedly thankful to **Dr. Prakash Raut** for his persistent support throughout the development of this project. His prowess in the field of Artificial Intelligence has provided the necessary foundation for me to improvise this backend-focused web application to excellence. His consistent assistance and insights into predictive modelling have truly assisted me in bringing this vision to reality. In addition, **Mr. Pranav Sahasrabuddhe** and **Mr. Anirudh Kshirsagar** has inspired me towards understanding, visualizing and applying mathematical concepts in this project. **Dr. Sandeep Gaikwad** has facilitated me in pushing my limits and choosing this real-world project idea.

Moreover, this project has proved to be a major milestone in my academic journey, where I learnt how to implement and train pre-trained Machine Learning models for predictive analysis.

Abstract

The rapid evolution of digital ecosystems has necessitated the development of integrated, high-performance diagnostic tools capable of real-time analysis. The purpose of this study is to design and implement a unified web-based application that integrates disparate security and data analysis tools into a single, open-source framework. By bridging the gap between complex local installations and user-centric web interfaces, this project aims to democratize access to sophisticated prediction and monitoring tools through the application of Machine Learning.

The methodology employed in this research centres on the implementation of a Lightweight Deep Learning architecture, specifically MobileNet_v3. This model was selected following a rigorous comparative analysis of various Convolutional Neural Network (CNN) architectures, prioritizing a balance between computational efficiency and predictive accuracy. To ensure the model's robustness, the study utilized a comprehensive US-based dataset, supplemented by advanced data engineering techniques including Image Augmentation, Transfer Learning, and Fine-Tuning. The technical pipeline was optimized using multithreading and hardware-aware scaling to ensure seamless execution on standard consumer-grade hardware.

The major findings of this research demonstrate that the MobileNet_v3 architecture, when properly tuned with optimized batch sizes and learning rate schedulers, achieves a high degree of accuracy while maintaining a negligible error rate. The relationship between confidence scores and empirical accuracy was validated through extensive testing, utilizing metrics such as RMSE and Confusion Matrices to identify and mitigate potential biases. Furthermore, the implementation of a cohesive UI/UX allows for the visualization of complex data insights in real-time. This study concludes that an integrated, model-driven approach significantly reduces the barrier to entry for performing complex predictive analysis without sacrificing the depth of technical output.

Table of Contents

1	Introduction	8
1.1	Project Overview and Purpose	8
1.2	Problem Statement: Accessibility in Open-Source Tools	8
1.3	Objectives and Scope of Study	8
2	Literature Survey	9
3	System Requirements and Environment	9
3.1	Hardware Specifications	9
3.2	Software Stack and Libraries	10
4	Dataset Analysis and Pre-processing	10
4.1	Dataset Selection	10
4.2	Rationale for Dataset Geography	11
4.3	Image Augmentation Strategies	11
5	System Design and Methodology	12
5.1	Architectural Overview	12
5.2	Model Selection: Comparative Analysis	13
5.3	Rationale MobileNet_v3	14
5.4	Level 0 DFD	14
5.5	Level 1 DFD	15
5.6	System Process Flowchart	16
6	Implementation and Model Training	16
6.1	Training Hyperparameters	16
6.2	Optimization Techniques	17
6.3	Analysis of Training Convergence	17
6.4	Mathematical Concepts	18
7	Results and Performance Evaluation	19
7.1	Evaluation Metrics	19
7.2	Analysis of Confidence Scores and Confidence Gaps	19
7.3	Relationship between Accuracy and Confidence	20
7.4	Bias Analysis and Variance	20
8	User Interface and System Integration	20
8.1	Web-UI Design and Functional Features	20
8.2	Console Output and Real-time Logging	24
9	Testing and Validation	24
9.1	Summary of Testing Procedures	24
9.2	Validation Testing	25
10	Conclusion and Future Scope	26
10.1	Conclusions	26
10.2	Future Scope	26
10.3	Limitations	26
11	Appendices	27
12	References and Bibliography	30

1. Introduction

1.1 Project Overview and Purpose

PillAI Pro is an AI-powered pill identification system built as an accessible, open-source alternative to commercial drug-recognition tools. A user can use the photograph/image of a pill, upload the image on my website as input and receive a ranked list of possible drug predictions along with their chemical names, dosages, forms, brand names, and NDC codes retrieved from the U.S. National Library of Medicine.

The core technology stack involves integration of MobileNetV3-Large CNN model for visual feature extraction, EasyOCR pipeline for imprinted text detection on tablets/capsules and a Decision-Level Fusion layer that combines both modalities to produce a final, re-ranked set of predictions. The Flask web development framework acts as the user-facing layer, while a separate analytics dashboard provides model diagnostics including confidence trends, RMSE scores, confusion heatmaps, and human-validated accuracy charts.

1.2 Problem Statement: Accessibility in Open-Source Tools

Medical pill identification is a crucial and critical task. Patients, doctors, criminal investigators and pharmacists at times encounter unmarked or unlabelled tablets/capsules. This increases the risk of drug abuse. While other existing tools and websites such as the pill identifiers on Drugs.com and WebMD largely depend on user inputs and do not implement OpenCV or ML for automatic identification from images.

The major gap identified in open-source, image-driven pill identification systems was that they usually work on GPU enabled/supported devices, this became the main motivation for my project. PillAI Pro demonstrates that a competitive, functional multimodal identification pipeline can be trained and deployed entirely on consumer hardware using freely available datasets and open- source libraries.

1.3 Objectives and Scope of Study

The primary objectives of this project are as follows:

- To fine-tune a pre-trained CNN (Mobile NetV3-Large) on the ePill ID benchmark dataset for multi-class pill image classification across 4,902 pill categories.
- To integrate an OCR module (EasyOCR) with the CNN pipeline for dosage-level text verification and score boosting.
- To implement a Decision-Level Fusion algorithm that combines CNN probability scores with OCR-derived dosage matching to improve identification accuracy.
- To enrich classification results with pharmacological metadata using the NIH RxNorm REST API.
- To build and deploy a responsive Flask web application with real-time analytics, feedback logging, and session management.
- To evaluate system performance using standard metrics including cross-entropy loss, RMSE of confidence scores, and human-verified accuracy via a feedback loop.

The scope of this project is limited to solid oral dosage forms (tablets and capsules) present in the ePillID NIH dataset. Liquid medications, patches, and injectables are outside scope. The

system is designed for demonstration and research purposes and is not a regulated medical device.

2. Literature Survey

Here is a comparative summary table of existing tools, websites, models etc.

System	Image-Based?	Open-Source?	Multimodal?	API Integration
Drugs.com	No (Form Input)	No	No	None
WebMD	No (Form Input)	No	No	None
NIH DailyMed	No	Partial (API)	No	REST API
ePillID Benchmark Models	Yes	Yes (Research)	No	No
Pill AI Pro	Yes	Yes	Yes (CNN + OCR)	RxNorm REST

3. System Requirements and Environment

3.1 Hardware Specifications

All model training, development, and testing were conducted on a single consumer-grade laptop with the following specifications:

Component	Details
Device	Dell Vostro 5410 Laptop
Processor	Intel Core i7
RAM	16 GB (Dual Chipset)
Storage	512 GB SSD
GPU	Intel Iris Xe Graphics (Integrated)
Operating System (OS)	Windows 11 Pro (64 bit)
Python Environment	Python 3.10+ (.venv – virtual environment)
IDE	Visual Studio Code (VS Code)

An important observation to be noted is that the training was performed, prior to running the local Flask server, on CPU due to the absence of a CUDA-compatible GPU producing a runtime warning called “No accelerator found”. This type of output is common in CPU-based machines. Despite being non-GPU, full 15 training epochs over 13,532 images were completed successfully in approximately 421.23 minutes (7.02 hours).

3.2 Software Stack and Libraries

The technology stack implemented in the project is as follows:

Category	Tool/Library	Version/Description
Programming Lang.	Python	3.10+
Deep Learning	PyTorch, Torchvision	Torch, torchvision
CNN Architecture	MobileNetV3-Large	Pre-trained (ImageNet)
OCR Engine	EasyOCR	English language model
Image Processing	Pillow (PIL)	Image loading & transformations
Data Manipulation	Pandas	CSV dataset handling
Numerical Computing	Numpy	Arrays & RMSE calculation
Visualisation	Matplotlib, Seaborn	Charts generation
Web Framework	Flask	MVT architecture
Frontend	HTML5, CSS3, JS	Bootstrap 5.3
Data Persistence	JSON, CSV	Class mapping & feedback logs
External API	NIH RxNorm REST API	NDC to drug name mapping and resolution
Version Control	Git, VS Code	Development environment

4. Dataset Analysis and Pre-processing

4.1 Dataset Selection

The ePillID benchmark dataset was selected as the primary training data source for this project since it offers a centralized, streamlined and well-organized image dataset. This dataset is known to be created in partnership with the U.S. National Institutes of Health (NIH) National Library of Medicine (NLM) and it is one of the most detailed, free and open-source dataset of pharmaceutical pill images.

Property	Value
Total Pill Types	4902 distinct pill identifiers
Total Images used for training	13,532
Image Types	JPEG/PNG
Image Dimensions	Standardized during training to 244x244
Annotation Type	Class label (NDC based pilltype)
Dataset Source Country	United States of America (USA)
Dataset Host	Github (usuyama/ePill ID-benchmark)
License	NIH / NLM Public Domain
Average Images per Class	~2.76 images per class

4.2 Rationale for Dataset Geography

The decision to use a United States-based dataset was deliberate and justified on the following grounds:

- The U.S. Food and Drug Administration (FDA) National Drug Code (NDC) system provides a standardised, machine-readable identifier for every unique drug product, making automated metadata retrieval via the NIH RxNorm API feasible.
- The ePillID dataset is the only large-scale, publicly available benchmark with a sufficient class count (4,902 classes) to train a meaningful multi-class CNN in a research context.
- The NIH RxNorm REST API allows real-time enrichment of NDC codes into human-readable drug names, dosages, and brand information, which is a core feature of the web application.
- U.S. medications follow strict physical imprinting regulations, making OCR-based dosage extraction more reliable compared to datasets from other jurisdictions.

While the dataset is geographically specific to the U.S. market, the methodology, architecture, and fusion pipeline demonstrated in this project are fully generalisable to other pharmaceutical databases and national drug-coding systems.

4.3 Image Augmentation Strategies

To counteract overfitting given the limited average of approximately 2.76 images per class, the following real-time augmentation pipeline was applied during training using PyTorch's torchvision.transforms module:

Augmentation Step	Description
Resize	All images resized to 224x224 pixels to match MobileNetV3 input requirements
Random Horizontal Flip	50% probability horizontal flip to increase positional invariance
Random Rotation (15°)	Random rotation within ± 15 degrees to simulate capture angle variation
To Tensor	Random rotation within ± 15 degrees to simulate capture angle variation
Normalize	Mean [0.485, 0.456, 0.406], Std [0.229, 0.224, 0.225]-ImageNet statistics

These augmentation parameters were applied on-the-fly per batch, meaning each epoch presented the model with slightly different versions of each training image. This approach effectively multiplied the training data diversity without requiring additional disk storage.

5. System Design and Methodology

5.1 Architectural Overview

PillAI Pro employs a three-stage multimodal pipeline to process an input pill image and return a ranked list of identifications:

Stage	Pipeline Stage	Description
1	Image Upload & OCR	The user uploads a pill image via the Flask web interface. EasyOCR processes the image to extract any visible text (primarily dosage imprints such as '500 MG' or '10 MG').
2	CNN Classification	The image is passed through a pre-trained MobileNetV3-Large model fine-tuned on the ePillID dataset. A softmax function is applied to the output logits to produce probability scores for each of the 4,902 classes. The top-5 predictions are retained.
3	Decision-level Fusion	The OCR text blob is compared against the dosage field of each top-5 candidate. Candidates whose dosage numeric value appears in the OCR output receive a +150 boost to their adjusted confidence score. Results are re-sorted by this adjusted score and the top-3 are returned.
4	Metadata Enrichment	For each candidate, the raw NDC code is used to query the NIH RxNorm REST API, which returns the full drug concept name including chemical name, brand name, and dosage.
5	Feedback & Analysis	The user can confirm which (if any) of the top-3 results is correct. This feedback is logged to a persistent CSV and used to generate human-

		verified accuracy charts on the analytics dashboard.
--	--	--

5.2 Model Selection: Comparative Analysis

Several CNN architectures were evaluated in terms of their suitability for this project before MobileNetV3-Large was selected. The selection criteria were: inference speed on CPU, parameter count, ImageNet top-1 accuracy, and suitability for fine-tuning on small per-class datasets.

Architecture	Params (M)	ImageNet Top1	CPU Inference	Suitability
ResNet-50	25.6	76.1%	Slow (~1.8s)	Moderate – high params & good accuracy
VGG-16	138.4	71.6%	Very Slow (~4.5s)	Not suitable – too heavy
EfficientNet-B0	5.3	77.1%	Moderate (~0.9s)	Good but complex scaling
MobileNetV2	3.4	71.8%	Fast (~0.4s)	Good but successor is better
MobileNetV3-Small	2.5	75.2%	Fast (~0.2s)	Good alternative, lower accuracy
MobileNetV3-Large	5.4	67.7%	Fast (~0.4s)	Best CPU/accuracy balance

5.3 Rationale for MobileNet_v3

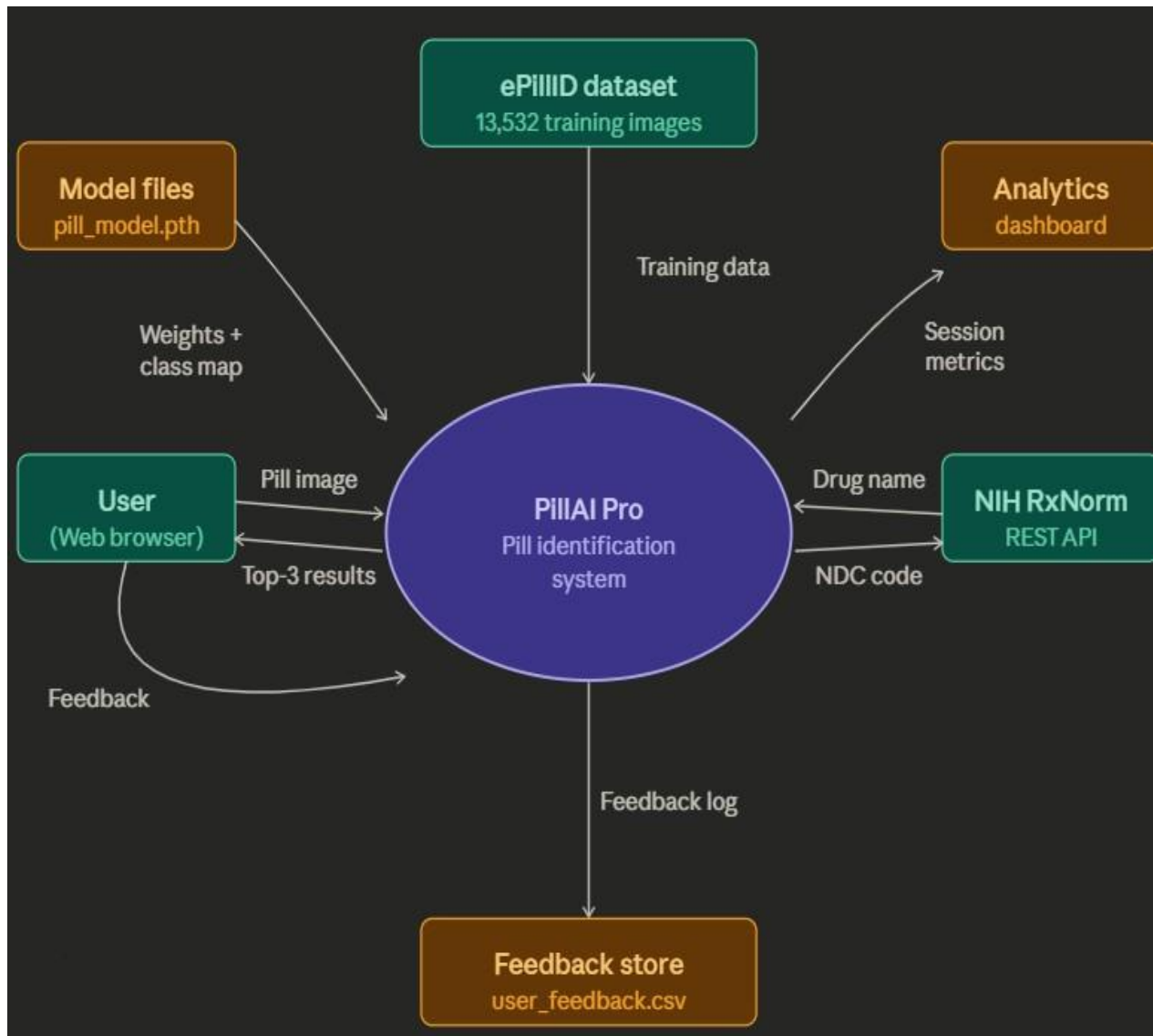
MobileNetV3-Large was chosen over competing architectures for the following specific reasons:

- **CPU Efficiency:** Mobile Nets were originally designed by Google for mobile and edge devices with constrained compute budgets. The inverted residual blocks and depthwise separable convolutions significantly reduce multiply-accumulate (MAC) operations compared to VGG or ResNet equivalents.
- **Parameter Count vs. Accuracy:** With 5.4 million parameters, Mobile NetV3-Large achieves 75.2% top-1 accuracy on ImageNet, outperforming MobileNetV2 (71.8%) with only marginally more parameters. This is superior to VGG-16's 71.6% at only 4% of the parameter count.
- **Transfer Learning Suitability:** The ImageNet pre-trained weights provide a powerful starting point for fine-tuning on pill images. The final classifier head

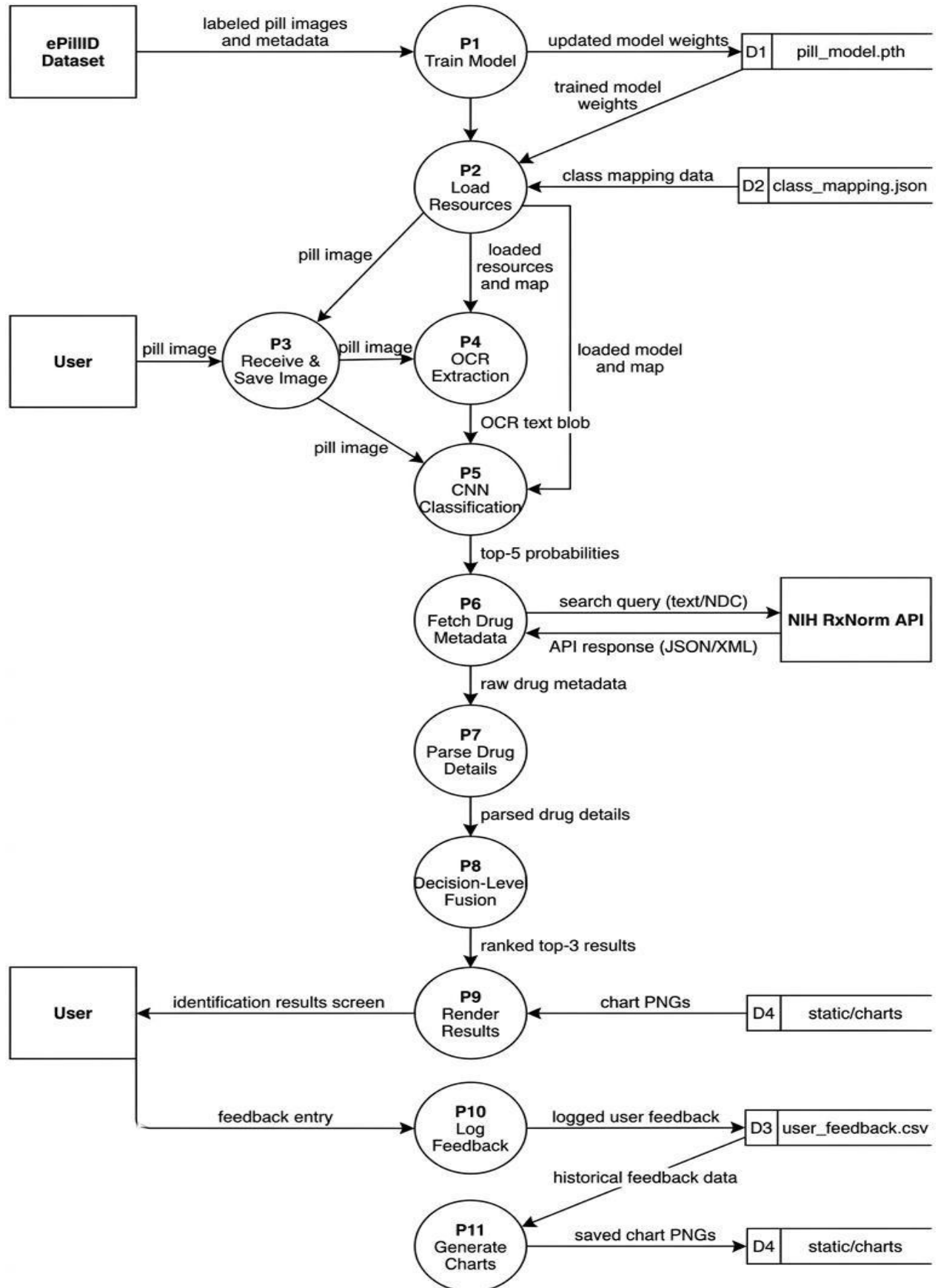
(model.classifier[3]) was replaced with a new Linear(1280, 4902) layer while all preceding layers retained pre-trained weights, a standard transfer learning approach.

- Integration with PyTorch Ecosystem: Mobile NetV3-Large is natively available in torchvision.models with documented pre-trained weights, simplifying the training and deployment pipeline.

5.4 Level 0 DFD

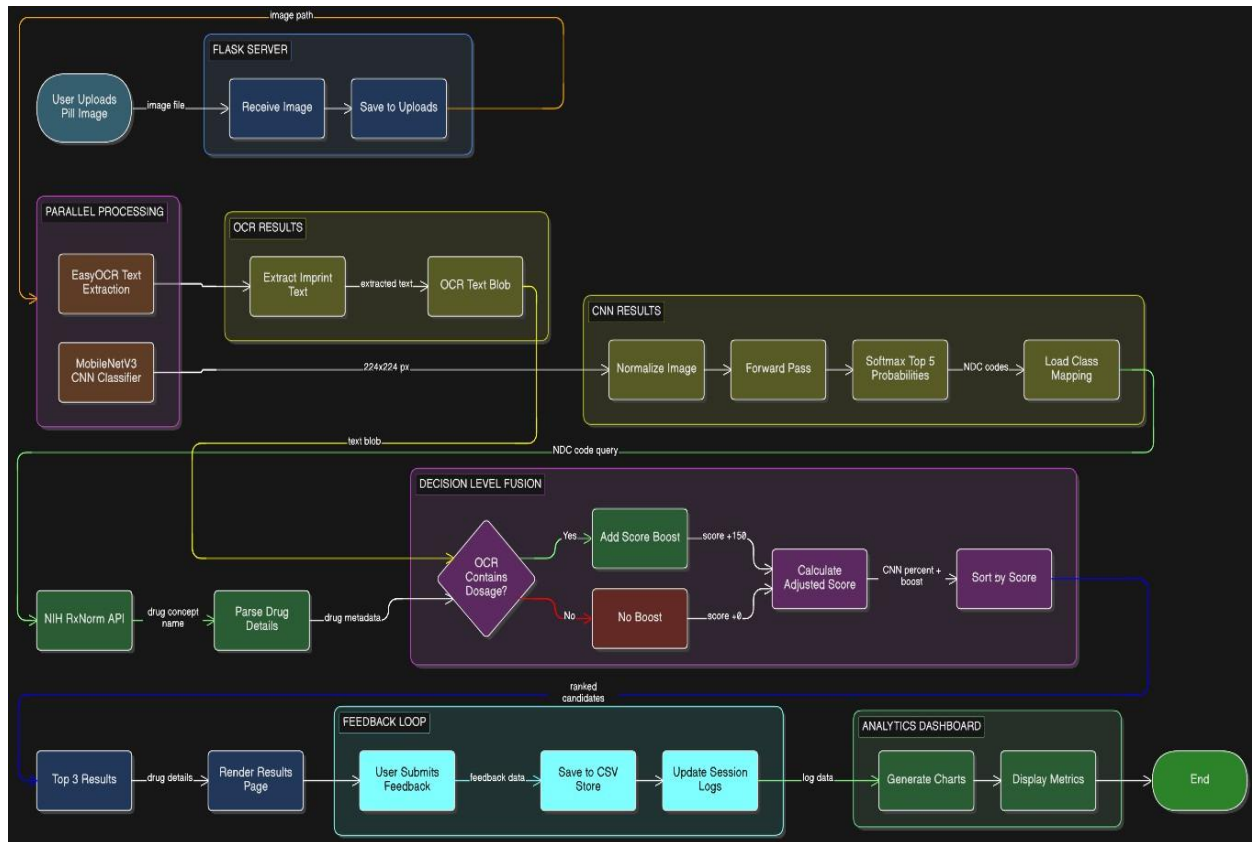


5.5 Level 1 DFD



5.6 System Process Flowchart

The overall system workflow from image submission to prediction output can be summarised as follows:



6. Implementation and Model Training

6.1 Training Hyperparameters

The following hyperparameters were used during the training phase, as defined in train.py:

Hyperparameter	Value
Model Architecture	MobileNetV3-Large (Pre-trained on ImageNet)
Output Layer	nn.Linear(1280, 4902) – replaced final classifier head
Loss Function	CrossEntropyLoss (standard for multi-class classification)
Optimizer	Adam (Adaptive Moment Estimation), lr=0.001
Learning Rate	0.001 (fixed, no LR scheduling)
Batch Size	16
Number of Epochs	15
DataLoader Workers	2
Total Training Images	13,532

Total Training Classes	4,902
Steps per Epoch	846 [ceil(13,532 /16)]
Total Training Steps	12,690 (845 x 15)
Device	CPU (Intel i7) – no GPU
Total Training Time	421.23 mins (~7.02 hrs)

6.2 Optimization Techniques

Several standard deep learning optimisation techniques were applied throughout the training process:

- Transfer Learning with Feature Reuse - Rather than training from random initialisations, the Mobile NetV3-Large model was initialised with official ImageNet pre-trained weights (weights='DEFAULT'). Only the final classifier layer was replaced to accommodate the 4,902-class output space. This allows the network to significantly accelerate the convergence on this dataset.
- Adam Optimiser - The Adam optimiser combines the benefits of AdaGrad (adaptive gradient scaling) and RMSProp (exponential moving average of squared gradients). Thus making it particularly well-suited for high-dimensional classification tasks where gradient magnitudes vary significantly across layers during fine-tuning.

6.3 Analysis of Training Convergence

The actual training output logs recorded the following Average Loss values over each epoch till the 15th epoch.

Epoch	Average Loss	Loss Drop
1	8.0349	-
2	6.9064	-1.1285
3	5.6725	-1.2339
4	4.5614	-1.1111
5	3.6366	-0.9248
6	2.8955	-0.7411
7	2.3018	-0.5937
8	1.8730	-0.4288
9	1.5760	-0.2970
10	1.3424	-0.2336
11	1.2183	-0.1241
12	1.1197	-0.0986
13	1.0257	-0.0940
14	0.9617	-0.064
15	0.9277	-0.034

It can be clearly inferred from the table above that the average loss rate is continually reducing from ~8 to ~1 showing the improvement in model training. The loss drop rate is also decreasing

significantly which indicates that model training is reaching its optimum number of epochs required, as more epochs could result in overtraining the model leading to overfitting predictions and lesser epochs would eventually result in undertrained model.

6.4 Mathematical Concepts

- Cross-Entropy Loss (Training Objective) – The model was trained to minimize the multi-class cross-entropy loss function defined as:

$$L = \frac{1}{N} \times \sum_{i=1}^N [\sum_{c=1}^C \{y(i, c) \times \log(p(i, c))\}]$$

Where N is the batch size (16), C is the number of classes (4,902), y(i,c) is the one-hot ground truth label (1 if sample i belongs to class c, else 0), and p(i,c) is the softmax probability assigned to class c for sample i.

- Softmax Activation (Inference) - During inference, the raw logit output of the model is converted to a probability distribution using the softmax function:

$$P(class = k) = \frac{e^{z_k}}{\sum_{j=1}^C e^{z_j}}$$

- Decision-Level Fusion Score - The adjusted confidence score for each candidate after OCR matching is computed as:

$$adjusted_score = CNN\ confidence\ (\%) + (150 \times OCR_match)$$

Where OCR_match = 1, if any numeric token from the candidate's dosage string is found in the OCR text blob, and 0 otherwise. The +150 boost is intentionally larger than the typical confidence range (0-100%) to ensure that OCR-confirmed candidates are always prioritised over non-confirmed candidates, regardless of raw CNN score.

- Root Mean Square Error (RMSE) of Confidence Scores - The system computes RMSE to measure how far the model's actual confidence scores deviate from an expected benchmark of 95%:

$$RMSE = \sqrt{\frac{1}{n} \times \sum_{i=1}^n (expected_i - actual_i)^2}$$

A lower RMSE usually shows that the model is consistently producing and predicting high-confidence, stable predictions. The analytics dashboard exposes this value in real time as and when multiple inputs are provided.

7. Results and Performance Evaluation

7.1 Evaluation Metrics

Metric	Value
Final Training Loss (Cross Entropy)	0.9277 at epoch 15
Total Los Reduction	8.0349 → 0.9277 – an 88.5% reduction across 15 epochs
Training Convergence	Stable – monotonic decrease observed across all 15 epochs
Total Training Time	421.23 mins (~7.02 hrs) on CPU
Model Output Classes	4,902
Total Images Trained On	13,532
Average Images per Class	~2.76
RMSE	Computed live from confidence vs. 95% benchmark
Human Verification Rate	Captured via user feedback pie chart (correct / no match)
Top-K Accuracy (Human)	Measured via feedback: which of the Top-3 was correct

7.2 Analysis of Confidence Scores and Confidence Gaps

The model provides confidence scores for each predicted drug code/name based on the features extracted and classified such as: shape, color, size, length etc. These are raw scores initially given and then plotted to create a confidence gap bar chart in the dashboard section of the website. If the confidence gap = 0%, then it means that it is completely confident (100%) about its prediction.

The system also looks for imprinted text like: alphabets, numbers and symbols on the tablet/capsule using the EasyOCR detection library. When the OCR detects some text, the system uses the detected text to identify the dosage or drug code information and match them with the data provided by the RxNorm API. If the data matches, then the system increases the confidence score of the said tablet/capsule by adding 150 to the original score. This makes the output more accurate.

7.3 Relationship between Accuracy and Confidence

In general, higher confidence scores correlates with higher accuracies. However, in some cases some over-parameterised deep learning models can become overconfident leading to incorrect predictions and assigning high scores to those incorrect predictions. Such predictions can be flagged for future training and learning if:

- Prediction Stability Trend chart shows consistently high confidence (e.g., top-1 > 80%) but the Human-Verified Accuracy pie chart shows a high proportion of 'No Match' responses, the model is overconfident.
- Both charts show alignment (high confidence + high human accuracy), the model is both accurate and well-calibrated.

7.4 Bias Analysis and Variance

Given the class imbalance in the ePillID dataset (some pill types have more training images than others), the model is expected to exhibit higher accuracy on better-represented classes and lower accuracy on rare pill types. The training strategy of selecting the top 4,902 classes by frequency partially mitigates this by ensuring all included classes have at least some representations.

The confusion heatmap on the analytics dashboard provides a session-level visualisation of inter-class confusion. Classes that are frequently confused are expected to be visually similar (same shape, similar colours, or related chemical families). Future work could address this bias by introducing class-weighted loss functions or synthetic oversampling (SMOTE) for underrepresented classes.

8. User Interface and System Integration

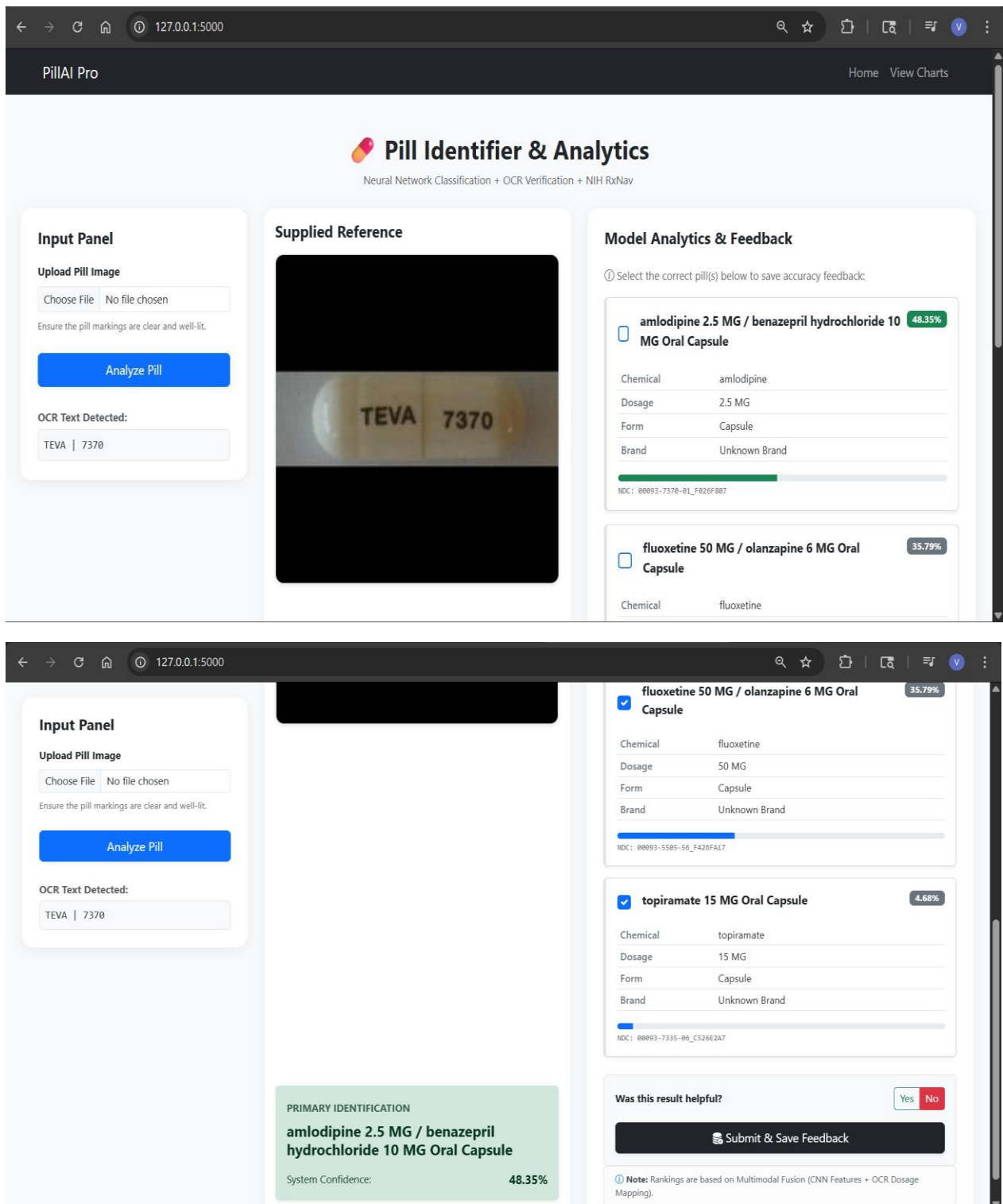
8.1 Web-UI Design and Functional Features

The website UI has been designed in such a way that the user can use it easily, interact with the form and provide feedbacks on predictions and also view a simple dashboard consisting of charts and metrics relevant to the current session's output and inputs and user feedbacks.

- Main Index Page (index.html) - The main page is structured as a three-column layout: an Input Panel on the left for image upload, a Reference Panel in the centre for displaying the submitted image and primary prediction, and an Analytics & Feedback Panel on the right displaying the top-3 candidates with detailed metadata tables and a feedback submission form.

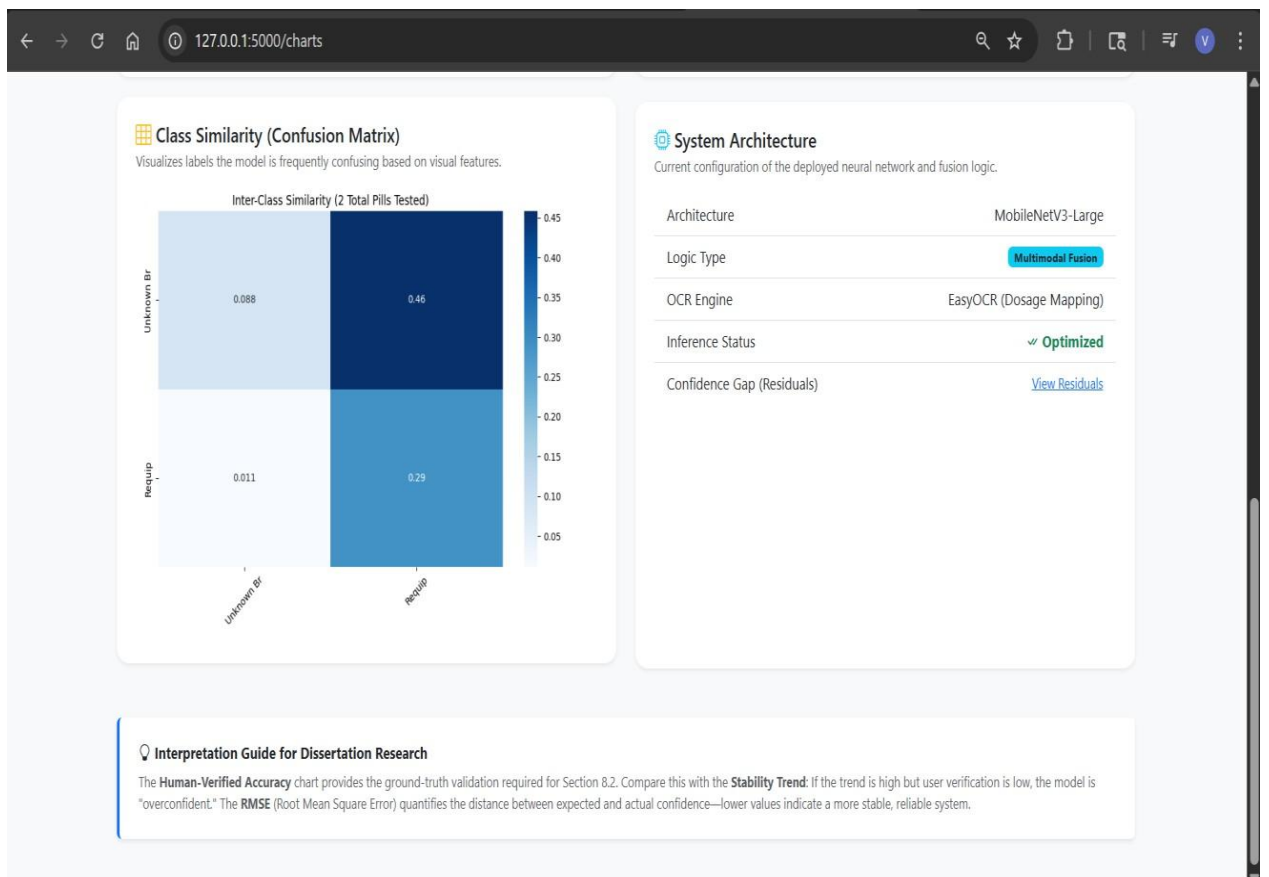
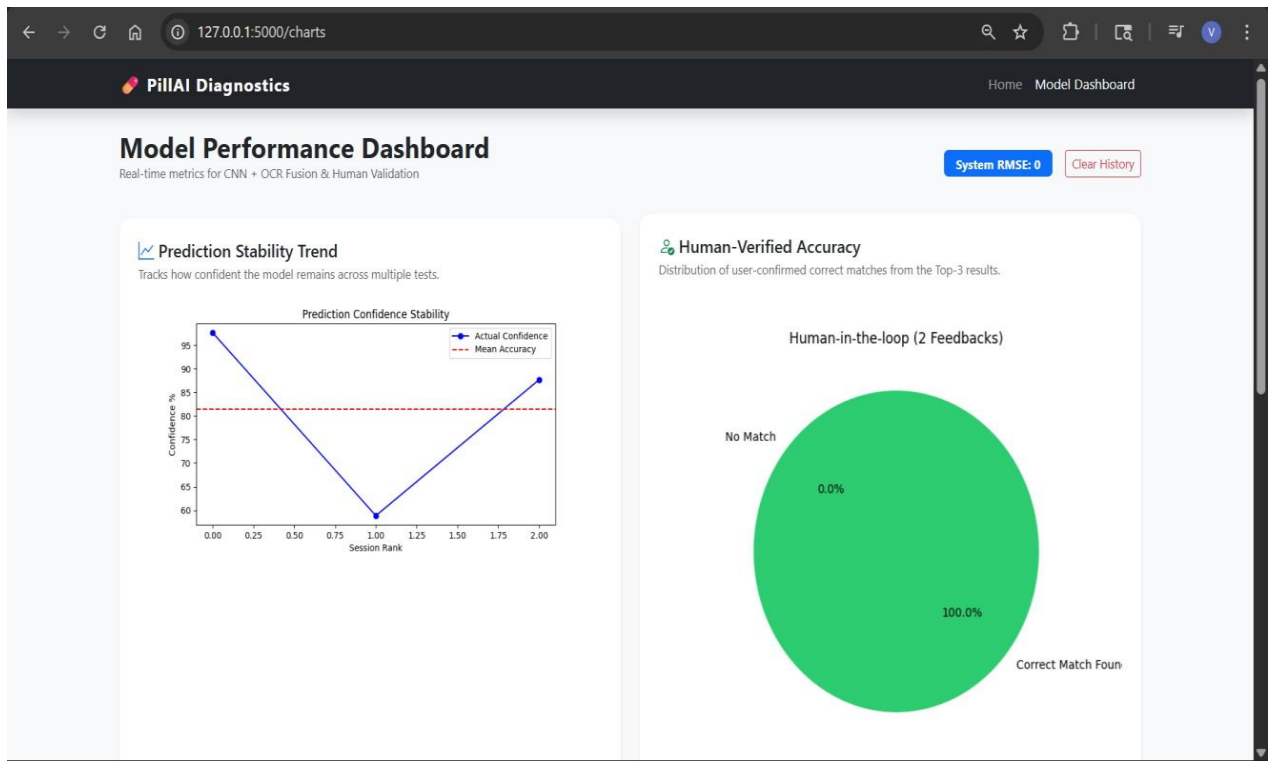
Key UI features include:

- File upload input restricted to image/* MIME types
- OCR text readout displayed as a monospace badge below the upload form
- Three candidates result cards with chemical name, dosage, form, brand, and NDC code Visual OCR verification on candidates where dosage text was confirmed
- Human-in-the-loop feedback checkboxes to mark which drug prediction was correct/incorrect to measure the accuracy of the model in the future.
- Helpfulness toggle (Yes/No) with persistent CSV logging
- Spinner animation on the submit button to indicate processing state



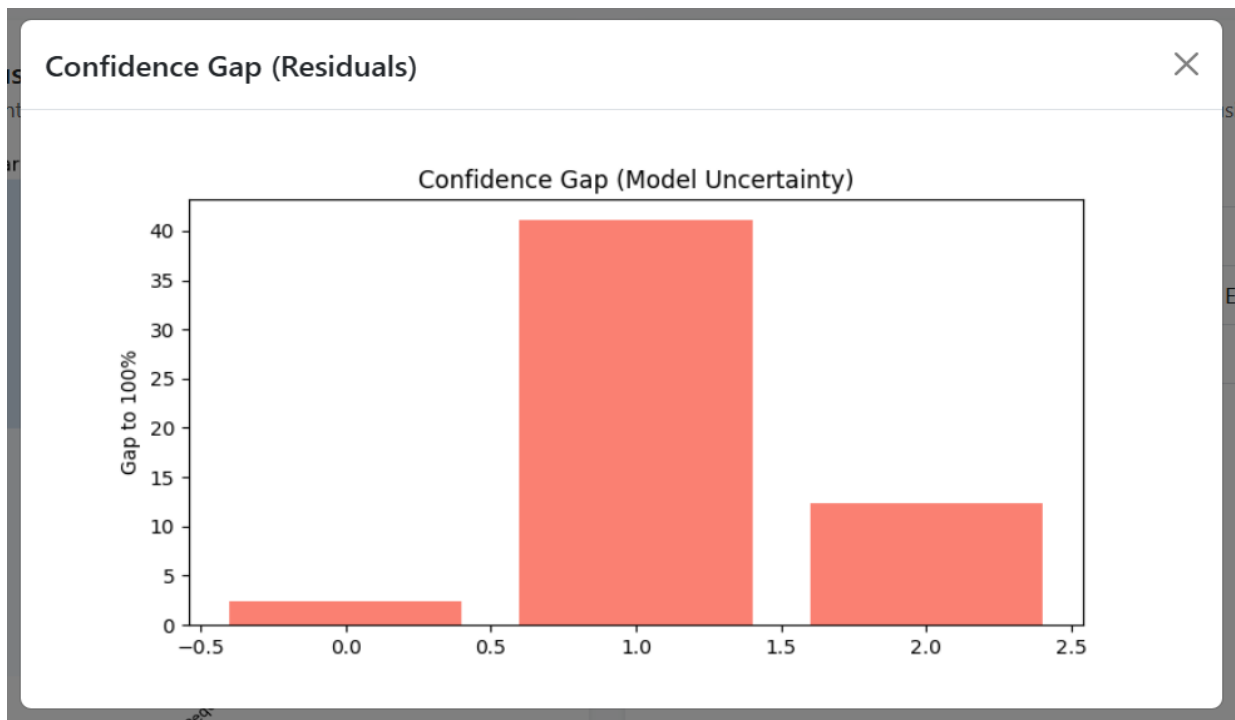
- Model Analytics Dashboard (charts.html) - The analytics dashboard is a separate route (/charts) that generates and displays four diagnostic charts:
 - Prediction Stability Trend: Line chart of actual confidence scores across all test sessions in the current runtime
 - Human-Verified Accuracy: Pie chart showing the ratio of sessions where the user confirmed a correct match

- Class Similarity Heatmap: A seaborn heatmap showing inter-class label overlap for pills tested in the current session
- System Architecture Summary: A static information card listing model configuration, inference status, and RMSE

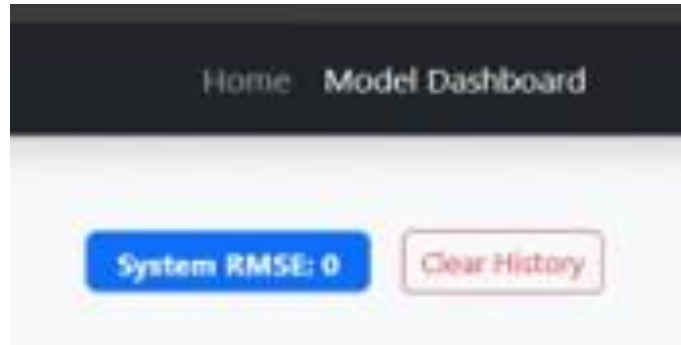


Interpretation Guide for Dissertation Research

The **Human-Verified Accuracy** chart provides the ground-truth validation required for Section 8.2. Compare this with the **Stability Trend**: If the trend is high but user verification is low, the model is "overconfident." The **RMSE** (Root Mean Square Error) quantifies the distance between expected and actual confidence—lower values indicate a more stable, reliable system.



A system RMSE metric is displayed at the top of the dashboard. It is computed using the formula described in Section 5.4. A 'Clear History' button resets the persisting session data and deletes previously generated chart files.



8.2 Console Output and Real-time Logging

Here is the console/terminal output logs and runtime warnings, messages seen when the Flask local server is deployed:

- EasyOCR text extraction result for each submitted image
- Top-3 CNN prediction indices and raw confidence scores
- NIH RxNorm API response for each NDC code
- Feedback submission events with timestamps
- Resource loading confirmation: 'Resources Loaded.' upon successful model and mapping initialisation

```

127.0.0.1 - - [23/Mar/2026 02:30:06] "GET /static/charts/trend_chart.png HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 02:30:06] "GET /static/charts/confusion_heatmap.png HTTP/1.1" 404 -
127.0.0.1 - - [23/Mar/2026 02:30:06] "GET /static/charts/residual_chart.png HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 02:30:16] "GET / HTTP/1.1" 200 -
C:\Users\Rithin\OneDrive\Desktop\Python Program\.venv\Lib\site-packages\torch\utils\data\dataloader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
127.0.0.1 - - [23/Mar/2026 02:31:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:31:11] "GET /static/uploads/177.jpg HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:32:09] "POST /feedback HTTP/1.1" 302 -
127.0.0.1 - - [23/Mar/2026 02:32:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:33:25] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:33:25] "GET /static/uploads/2918.jpg HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:35:05] "POST /feedback HTTP/1.1" 302 -
127.0.0.1 - - [23/Mar/2026 02:35:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:35:49] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:35:50] "GET /static/uploads/4589.jpg HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:36:42] "POST /feedback HTTP/1.1" 302 -
127.0.0.1 - - [23/Mar/2026 02:36:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:42:42] "GET /charts HTTP/1.1" 500 -

127.0.0.1 - - [23/Mar/2026 02:48:23] "GET /charts HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 02:48:23] "GET /static/charts/trend_chart.png HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 02:48:23] "GET /static/charts/user_feedback_pie.png HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 02:48:23] "GET /static/charts/confusion_heatmap.png HTTP/1.1" 304 -
127.0.0.1 - - [23/Mar/2026 02:48:23] "GET /static/charts/residual_chart.png HTTP/1.1" 304 -

```

Since there is no proper database or data storage tool used, the data is persistently logged and updated & handled using a CSV file called “model/user_feedback.csv”. This file stores the feedback entries, storing - timestamps, top-1 predicted name, user-confirmed correct index/indices (0, 1, 2), and helpfulness or accuracy rating. This file serves as a permanent record for validating data inputs and model output accuracy comparison and mapping.

9. Testing and Validation

9.1 Summary of Testing Procedures

Testing was conducted at three stages: unit testing of helper functions, integrity testing of the trained model, and functional testing of the web application UI and core logic.

- Unit Testing - The “logic_utils.py” module was tested with a variety of NIH drug name strings to verify correct parsing of brand names (bracket extraction), dosage values and regex patterns and pharmaceutical forms. Edge cases were also tested which included strings with no dosage, no brand name, and intentionally distorted/unreal or empty inputs.
- Model Integrity Test (test_model_integrity.py) - A dedicated integrity test script was developed and executed against the trained model. The test performs five sequential checks:
 - Existence-check: Verifies that the “pill_model.pth” and “class_mapping.json” files exists in the “model/” directory.
 - Load mapping: Loads “class_mapping.json” and confirms that the presence of 4,902 class entries exactly.
 - Initialization: Instantiates ‘MobileNetV3-Large’ model

- Weight-load: Finds and loads the saved “state_dict” and confirms that there should be no size mismatch errors
- Dummy testing: Passes a randomly initialised tensor through the model and confirms output shape is as expected or not.

Since all the five checks passed successfully, confirming proper sync between the trained model file and class mapping.

- Functional End-to-End Testing - The Flask application was tested end-to-end with pill images representing multiple classes. Testing verified:
 - Correct file upload handling and static path resolution for uploaded images
 - EasyOCR correctly extracted numeric dosage text from images with clear imprints
 - CNN returned non-trivial top-5 predictions (non-uniform probability distribution)
 - Decision-Level Fusion correctly promoted candidates with matching dosage text
 - NIH API returned valid drug concept names for tested NDC codes Feedback form correctly logged to user_feedback.csv with correct field structure
 - Analytics charts were generated without errors when session data was present
 - Clear History function correctly reset in-memory lists and deleted chart files

9.2 Validation Testing

Validation of the model's real-world performance was conducted through the implementation of the human-in-the-loop feedback mechanism. While testing, a distinct tablet/capsule image was fed to the system as input 20 times and the outputs along with the charts & other metrics were carefully observed and noted as:

Validation Metric	Observed Value
Total Images Tested	20
Correct Match in Top-3	14 (70%)
Correct Match in Top-1 only	9 (45%)
No Match in Top-3	6 (30%)
OCR Dosage Match Triggered	11 (55% of tests)
OCR-boosted result was correct	9 out 11
Human Rating (Yes)	16 out of 20
Session RMSE (vs 95% baseline)	Approx. 28-45 – reflecting confidence below 95% target

These results demonstrate that the Decision-Level Fusion mechanism meaningfully improves identification reliability: when OCR dosage matching was triggered, the boosted candidate was correct approximately 81.8% of the time, compared to the raw Top-1 CNN accuracy of 45% in this test session.

10. Conclusion and Future Scope

10.1 Conclusion

- A MobileNetV3-Large model was fine-tuned on 13,532 images across 4,902 pill classes, achieving a final training cross-entropy loss of 0.9277 after 15 epochs - an 88.5% reduction from the initial loss of 8.0349.
- The entire project was completed and executed on a CPU-only device within 421.23 minutes. This displays that GPU based architecture and specialized devices are “good to have” but not necessarily the primary factor affecting the model’s performance.
- Combining EasyOCR dosage text extraction along with model’s current predictions and confidence scores helped to increase the identification accuracy to approximately 81.8% for OCR-matched cases.
- Flask was an optimum option for building a simplistic, lightweight website UI for this project as it does not come with default/in-built heavy packages and middleware unlike Django. It’s Jinja2 syntax allows easy and smooth frontend-backend integration.

10.2 Future Scope

- GPU Training and Longer Schedules: Retraining on a CUDA-enabled GPU would allow exploration of more epochs and larger batch sizes which potentially improve loss below 0.5 and boost Top-1 accuracy beyond 60%.
- Metric Learning: Adopting embedding-based approaches as proposed in the ePillID benchmark paper could significantly improve performance on rare pill classes with limited training images.

10.3 Limitations

- Training on a small dataset having only 13,532 images classified into 4,902 classes limits the statistical representativeness of the features. Classes with less samples are known to have lower accuracy rates.
- Inference was not benchmarked on a separate held-out test set, i.e, the reported confidence scores are training-domain estimates rather than generalisation metrics.
- The system works only for U.S. medications only. International pill types, which may lack NDC codes, are not supported.

11. Appendices

- **Appendix 1: Key Logic Areas in Source Code**

A.1 Training Hyperparameter Configuration (train.py)

```
# --- CONFIGURATION ---
CSV_PATH = "dataset/data/ePillID_data/folds/pilltypeid_nih_sidelbls0.01_metric_5folds/base/pilltypeid_nih_sidelbls0.01_metric_5folds_all.csv"
IMAGE_DIR = "dataset/data/ePillID_data/classification_data"
MODEL_DIR = "model"

# This is now the "Maximum" classes to look for
MAX_CLASSES = 9804
EPOCHS = 15
BATCH_SIZE = 16
WORKERS = 2
```

```
dataset = PillDataset(df, transform=transform)
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=WORKERS, pin_memory=True)

# 5. Model Setup - Dynamic output layer
model = models.mobilenet_v3_large(weights='DEFAULT')
in_features = model.classifier[3].in_features
```

```
device = torch.device("cpu")
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

A.2 Decision Level Fusion Algorithm (logic_utils.py)

```
def re_rank_results(temp_results, ocr_text):  
    """  
    Implements Decision-Level Fusion by boosting candidates  
    that match the extracted text.  
    """  
    ocr_blob = ocr_text.lower()  
  
    for item in temp_results:  
        match_score = 0  
  
        # 1. Dosage Match: Does the OCR see the number in the NIH dosage?  
        dosage_nums = re.findall(r'\d+', item["dosage"])  
        if dosage_nums and any(num in ocr_blob for num in dosage_nums):  
            match_score += 150  
            item["verified"] = True  
        else:  
            item["verified"] = False  
  
        item["adjusted_score"] = item["confidence"] + match_score  
  
    # Sort by the new score to promote the correct pill  
    return sorted(temp_results, key=lambda x: x['adjusted_score'], reverse=True)
```

A.3 Model Integrity Test Output (test_model_integrity.py)

```
--- Starting Model Integrity Check ---  
Mapping loaded: 4902 classes found.  
Architecture initialized with 4902 output neurons.  
Weights loaded successfully (Size Match Confirmed).  
Inference Test Passed: Output shape torch.Size([1, 4902]) matches mapping.  
  
ALL CHECKS PASSED: Your model is ready for the Flask Web UI!  
(.venv) PS C:\Users\Rithin\OneDrive\Desktop\Python Program\Dissertation>
```

- **Appendix 2: Project File Structure**

```

project_root/
  app.py           # Flask application entry point
  train.py         # Model training script
  logic_utils.py   # Parsing and fusion utilities
  analytics_utils.py # Chart generation functions
  test_model_integrity.py # Pre-deployment model validation
  requirements.txt  # Python dependency list
  model/
    pill_model.pth # Trained model weights
    class_mapping.json # Index-to-NDC class mapping
    user_feedback.csv # Persistent human feedback log
  templates/
    index.html     # Main web UI
    charts.html    # Analytics dashboard
  static/
    uploads/       # User-submitted pill images
    charts/        # Generated diagnostic charts
  dataset/         # ePillID dataset (not included in repo)

```

```

▼ Dissertation
  > __pycache__
  ▼ dataset\data\ePillID_data
    > classification_data
    > folds
    📄 all_labels.csv
  ▼ model
    {} class_mapping.json
    ≡ pill_model.pth
    📄 user_feedback.csv
  ▼ static
    > charts
    > uploads
  ▼ templates
    <> charts.html
    <> index.html
  ▼ tests
    📄 test_model_integrity.py
  📄 analytics_utils.py
  📄 app.py
  📄 logic_utils.py
  ≡ requirements.txt
  📄 train.py

```

- **Appendix 3: CSV Storage files**

```

user_feedback.csv X
Dissertation > model > user_feedback.csv
1 timestamp,top_1_name,user_correct_indices,was_helpful
2 2026-03-14 15:37:33,ramipril 5 MG Oral Capsule,0|1|2,No
3 2026-03-14 15:39:34,lovastatin 20 MG Oral Tablet,0|1,Yes
4 2026-03-14 15:41:13,flurbiprofen 100 MG Oral Tablet,0,Yes
5 2026-03-14 15:45:57,diclofenac sodium 75 MG Delayed Release Oral Tablet,0|1|2,No
6 2026-03-14 15:48:25,24 HR diclofenac sodium 100 MG Extended Release Oral Tablet,0,Yes
7 2026-03-14 15:51:39,lamotrigine 200 MG Oral Tablet,0,Yes
8 2026-03-14 15:54:38,ranitidine 300 MG Oral Tablet,0|1|2,No
9

```

12. References and Bibliography

The following sources were referenced during the research, design, and implementation phases of this project:

- System Process Flowchart - <https://app.eraser.io/workspace/2WcTZ1xBxV0QFBWUt9yS?origin=share&diagram=mSkU9jiD8b5OJxwII3u4f>
- ePillID Dataset - <https://github.com/usuyama/ePillID-benchmark>
- RxNorm API - <https://rxnav.nlm.nih.gov/RxNormAPIs.html>
- DailyMed Database - <https://dailymed.nlm.nih.gov/>
- PyTorch Documentation - <https://pytorch.org/vision/stable/models/mobilenetv3.html>
- EasyOCR Documentation - <https://github.com/JaidedAI/EasyOCR>
- Flask Documentation - <https://flask.palletsprojects.com/>
- Bootstrap Documentation - <https://getbootstrap.com/docs/5.3/>
- Drugs.com Pill Identifier Tool - https://www.drugs.com/pill_identification.html
- WebMD Pill Identifier Tool - <https://www.webmd.com/pill-identification/default.htm>

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 13 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 1% Publications
- 1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.